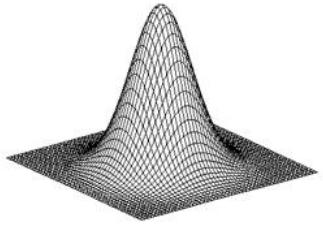
CS5187: Vision & Image

香港城市大學(東莞)
City University of Hong Kong
(Dongguan)

Zhu Zhiyu

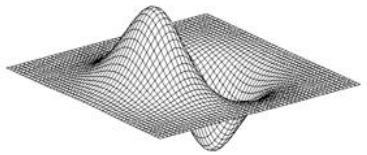
Tutorial 02: Edge Detection & Image Resampling

（1）Visualize the Gaussian Kernel and its derivative



Gaussian

$$h_\sigma(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



derivative of Gaussian (x)

$$\frac{\partial}{\partial x} h_\sigma(u,v)$$

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def generate_gaussian_plots():
    # 1. Setup the grid
    # Creating a coordinate grid from -3 to 3
    x = np.linspace(-3, 3, 100)
    y = np.linspace(-3, 3, 100)
    X, Y = np.meshgrid(x, y)

    # 2. Define Parameters
    sigma = 1.0

    # 3. Calculate Gaussian (h_sigma)
    # Formula: (1 / (2*pi*sigma^2)) * exp(-(u^2 + v^2) / (2*sigma^2))
    gaussian = (1 / (2 * np.pi * sigma**2)) * np.exp(-(X**2 + Y**2) / (2 * sigma**2))

    # 4. Calculate Derivative with respect to x (u direction)
    # Analytical derivative: -x / sigma^2 * Gaussian
    derivative_x = -(X / sigma**2) * gaussian

    # 5. Visualization
    fig = plt.figure(figsize=(14, 6))

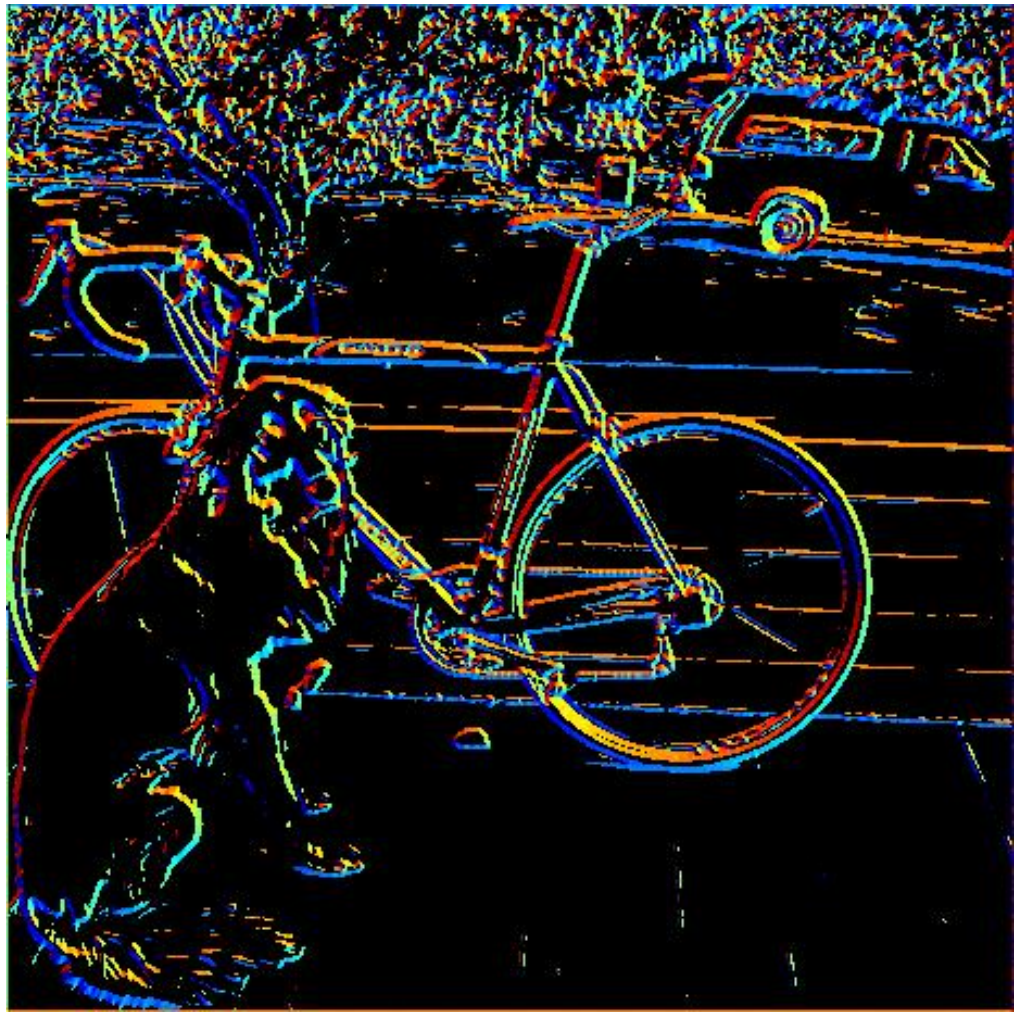
    # Plot 1: The Gaussian
    ax1 = fig.add_subplot(1, 2, 1, projection='3d')
    surf1 = ax1.plot_surface(X, Y, gaussian, cmap='gray', edgecolor='none', rstride=3, cstride=3, alpha=0.9)
    ax1.set_title(r"Gaussian $h_\sigma(u, v)$", fontsize=14, y=1.02)
    ax1.set_zlim(0, np.max(gaussian))
    ax1.axis('off') # Turn off axes for a cleaner textbook look

    # Plot 2: The Derivative of Gaussian (DoG)
    ax2 = fig.add_subplot(1, 2, 2, projection='3d')
    surf2 = ax2.plot_surface(X, Y, derivative_x, cmap='gray', edgecolor='none', rstride=3, cstride=3, alpha=0.9)
    ax2.set_title(r"Derivative of Gaussian $\frac{\partial}{\partial x} h_\sigma(u, v)$", fontsize=14, y=1.02)
    ax2.set_zlim(np.min(derivative_x), np.max(derivative_x))
    ax2.axis('off')

    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    generate_gaussian_plots()
```

（2）Sobel filtering the edges in the image



$$\theta = \text{atan2}(gy, gx)$$




```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

def visualize_gradient_orientation(image_path):
    # 1. Load the image as grayscale
    # We need grayscale to calculate the derivative of intensity
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print("Error: Could not load image.")
        return

    # 2. Calculate the Derivatives (Gradients) using Sobel
    # cv2.CV_64F allows for negative values (which are crucial for direction)
    gX = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=11)
    gY = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=11)

    # 3. Convert Cartesian (x, y) to Polar (magnitude, angle)
    # magnitude: The strength of the edge
    # angle: The direction of the edge (0 to 360 degrees)
    magnitude, angle = cv2.cartToPolar(gX, gY, angleInDegrees=True)

    # 4. Create the HSV Image
    # We create a blank image with 3 channels (Hue, Saturation, Value)
    hsv = np.zeros((img.shape[0], img.shape[1], 3), dtype=np.uint8)

    # --- Channel 0: Hue (Color) ---
    # OpenCV Hue range is 0-179. Angles are 0-360.
    # We divide by 2 to fit the angle into the Hue channel.
    hsv[..., 0] = angle / 2

    # --- Channel 1: Saturation ---
    # Set to 255 for maximum color vibrancy
    hsv[..., 1] = 255

    # --- Channel 2: Value (Brightness) ---
    # We normalize the magnitude to 0-255 so edges are bright and flat areas are black.
    # cv2.normalize handles the scaling automatically.
    hsv[..., 2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)

    # 5. Convert HSV back to BGR (for display/saving)
    # This creates the final RGB-like image you see in your example
    colored_derivative = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    # Display results
    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.title("Original Grayscale")
    plt.imshow(img, cmap='gray')
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.title("Gradient Orientation (Colorized)")
    plt.imshow(cv2.cvtColor(colored_derivative, cv2.COLOR_BGR2RGB))
    plt.axis('off')

    plt.show()

visualize_gradient_orientation('./example_image_02.jpg')
```

（2）Sobel filtering the edges in the image

Original Grayscale



Gradient Orientation (Colorized)



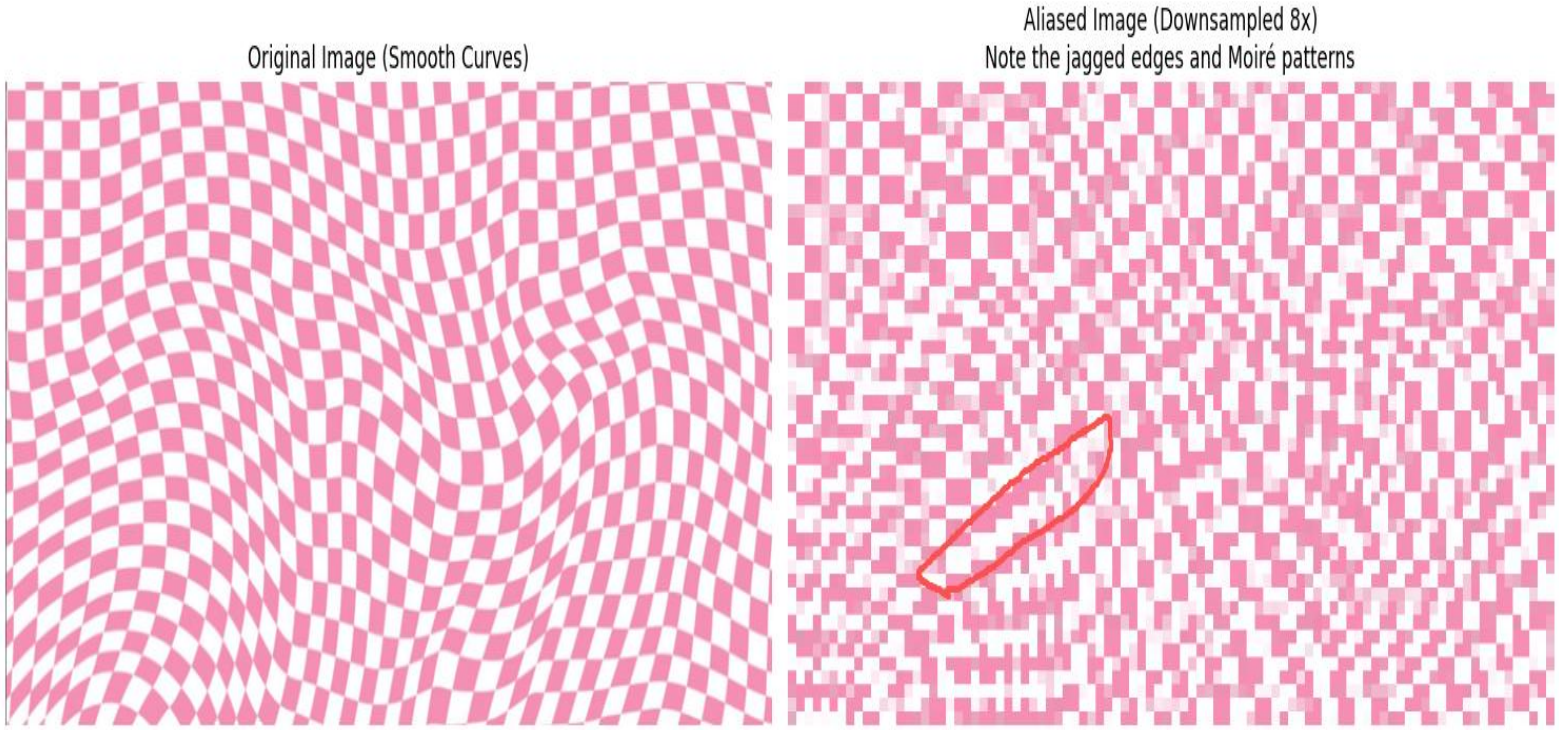
（3）Question: visualize the edge orientation

Edge Orientation V.S. Gradient Orientation





（4）Image Resampling: Visualizing Aliasing




```python
import matplotlib.pyplot as plt
from PIL import Image

def create_aliasing_effect(image_path, factor=5):
    """
    Introduces aliasing by downsampling without a filter (Nearest Neighbor)
    and then upsampling back to original size to visualize the artifacts.
    """
    try:
        # Load the original image
        img = Image.open(image_path)
        original_width, original_height = img.size

        # Calculate new "low res" dimensions
        # We shrink the image significantly to force sampling errors
        small_width = original_width // factor
        small_height = original_height // factor

        # Step 1: Downsample using Nearest Neighbor
        # This is where the aliasing happens. We act as if we are sampling
        # the signal at a rate too low for the frequency of the checks.
        img_small = img.resize((small_width, small_height), resample=Image.NEAREST)

        # Step 2: Upsample back to original size
        # We do this so you can compare it side-by-side with the original.
        # We use NEAREST again to keep the "blocky" aliased look sharp.
        img_aliased = img_small.resize((original_width, original_height), resample=Image.NEAREST)

        # Visualization
        fig, axes = plt.subplots(1, 2, figsize=(15, 8))

        # Original
        axes[0].imshow(img)
        axes[0].set_title("Original Image (Smooth Curves)")
        axes[0].axis('off')

        # Aliased
        axes[1].imshow(img_aliased)
        axes[1].set_title(f"Aliased Image (Downsampled {factor}x)\nNote the jagged edges and Moiré patterns")
        axes[1].axis('off')

        plt.tight_layout()
        plt.show()

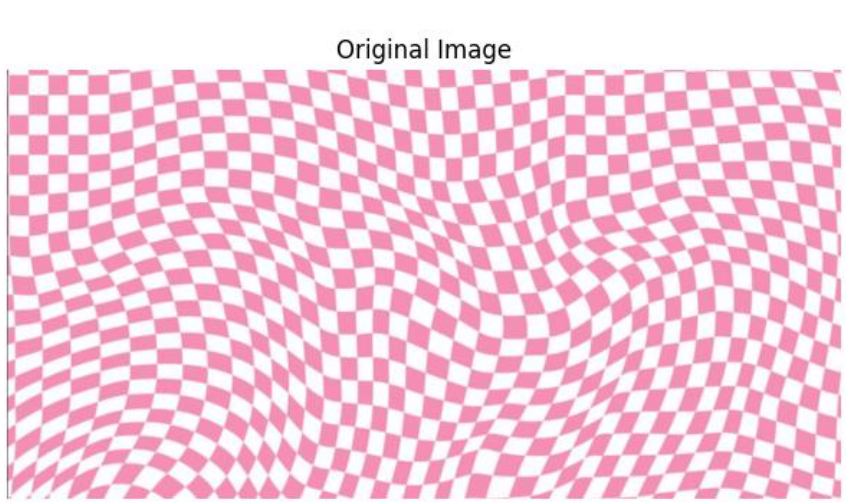
        return img_aliased

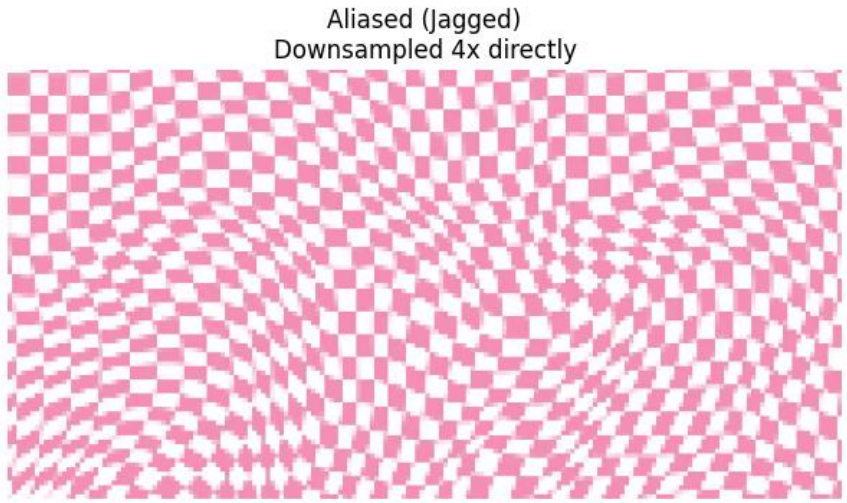
    except FileNotFoundError:
        print(f"Error: The file '{image_path}' was not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

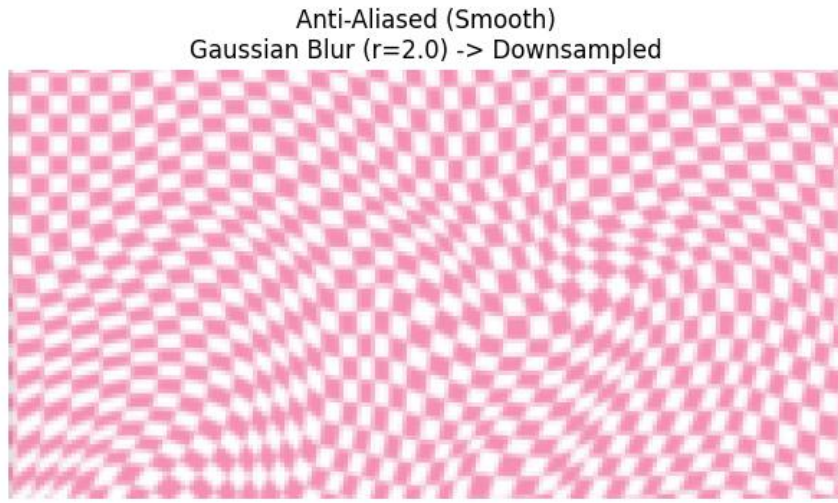
# Run the function
# You can increase the 'factor' to make the effect more extreme
create_aliasing_effect('./Example_Image_04.png', factor=8)
```

香港城市大學（東莞）
City University of Hong Kong
(Dongguan)

（5）Robust Resampling with Gaussian Blur







（5）Robust Resampling with Gaussian Blur

```python
import matplotlib.pyplot as plt
from PIL import Image, ImageFilter

def create_antialiasing_effect(image_path, factor=4):
    """
    Demonstrates Anti-Aliasing by applying a Gaussian Blur before downsampling.
    """
    try:
        # Load the original image
        img = Image.open(image_path)
        original_width, original_height = img.size

        # Calculate new "low res" dimensions
        small_width = original_width // factor
        small_height = original_height // factor

        # --- 1. Create the ALIASED version (Bad) ---
        # Just downsample directly with Nearest Neighbor.
        # This causes the jagged edges and Moiré patterns.
        img_aliased_small = img.resize((small_width, small_height), resample=Image.NEAREST)
        img_aliased = img_aliased_small.resize((original_width, original_height), resample=Image.NEAREST)

        # --- 2. Create the ANTI-ALIASED version (Good) ---
        # Step A: Low-pass filter (Gaussian Blur).
        # We blur the image to remove high frequencies that the small resolution can't handle.
        # A radius roughly half the downsampling factor is a good rule of thumb.
        blur_radius = factor/2
        img_blurred = img.filter(ImageFilter.GaussianBlur(radius=blur_radius))

        # Step B: Downsample the blurred image.
        img_aa_small = img_blurred.resize((small_width, small_height), resample=Image.NEAREST)

        # Step C: Upsample back for visualization
        img_antialiased = img_aa_small.resize((original_width, original_height), resample=Image.NEAREST)

        # --- Visualization ---
        fig, axes = plt.subplots(1, 3, figsize=(18, 6))

        # Plot Original
        axes[0].imshow(img)
        axes[0].set_title("Original Image")
        axes[0].axis('off')

        # Plot Aliased
        axes[1].imshow(img_aliased)
        axes[1].set_title(f"Aliased (Jagged)\nDownsampled {factor}x directly")
        axes[1].axis('off')

        # Plot Anti-Aliased
        axes[2].imshow(img_antialiased)
        axes[2].set_title(f"Anti-Aliased (Smooth)\nGaussian Blur (r={blur_radius}) -> Downsampled")
        axes[2].axis('off')

        plt.tight_layout()
        plt.show()

    except FileNotFoundError:
        print(f"Error: The file '{image_path}' was not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

# Run the function
# Using a factor of 4 makes the difference very obvious
create_antialiasing_effect('./Example_Image_04.png', factor=4)
```

Questions?

香港城市大學（東莞）
City University of Hong Kong
(Dongguan)